

MedAssitAI: Medical Symptom Analysis & Disease Prediction System

Milestone 4 Deployment Documentation

Submitted By

Name: Diksha

Team: 2

Role: Frontend

Project Overview

Milestone 4 focuses on understanding and documenting the deployment of the MedAssistAI project. The application developed during the previous milestones was moved from the local development environment to a publicly accessible deployment environment. The team deployed the frontend using Vercel and the backend using Render.

As a frontend team member, my focus in this milestone is on understanding how the frontend is prepared for production, how it communicates with the deployed backend, and how the final application can be accessed and tested through the deployed environment. The deployment itself was carried out as a team activity.

1. Deployment – Basic Concept

Deployment is the process of making a software application available in an environment where users can access and use it. During development, an application is usually run on a local computer using a local development server. Deployment moves the application from this local environment to a server or cloud platform.

For a web application, deployment generally involves preparing the project files, installing the required dependencies, configuring environment variables, building the application when required, starting the required services, and making the application available through a public URL.

In MedAssistAI, deployment separates the user interface from the server-side services. The frontend is made available through Vercel, while the backend is hosted through Render. The two parts communicate through the application's API.

2. Deployment Architecture of MedAssistAI

The deployed MedAssistAI system follows a frontend-backend architecture. The user interacts with the frontend through a web browser. The frontend sends the required requests to the backend through API communication, and the backend processes the requests and returns the required response.

The overall deployment flow can be represented as:

User → Vercel Frontend → Backend API on Render → Response → Vercel Frontend

This architecture allows the frontend and backend to be maintained and deployed as separate services while still working together as one application.

3. Frontend Deployment Using Vercel

Vercel is a cloud platform commonly used for deploying frontend and web applications. For the MedAssistAI project, Vercel is used to make the frontend available through a public URL.

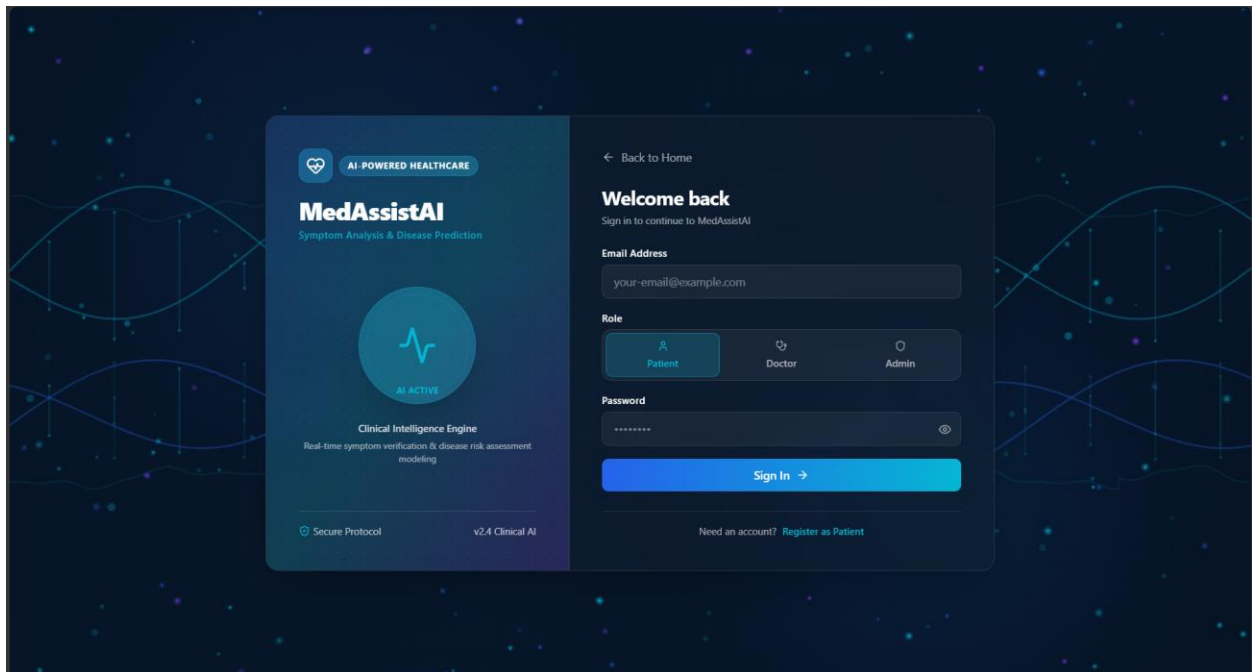
General Deployment Process

- The frontend project is maintained in the project repository.
- The repository is connected to the Vercel project.
- The appropriate project settings and build configuration are selected.
- Required environment variables are configured when needed.
- Vercel builds the project and creates a deployment.
- After a successful deployment, the frontend can be accessed through the assigned URL.

The team's deployed frontend is available at:

<https://medassistai-frontend.vercel.app>

The following screenshot shows the deployed MedAssistAI frontend. The application is accessible through the production environment and presents the authentication interface to the user.



4. Backend Deployment Using Render

Render is a cloud platform that can be used to host backend services and web applications. In the MedAssistAI project, the backend component was deployed by the team using Render.

General Deployment Process

- The backend project is maintained in the project repository.
- The backend repository or service is connected to Render.
- The required runtime and dependencies are configured.
- The build and start commands are configured according to the backend implementation.

- Required environment variables are added through the deployment platform.
- Render builds and starts the backend service.
- The deployed backend provides an address that can be used by the frontend for API communication.

The exact backend URL and service configuration depend on the team's Render deployment. The backend URL is not included here because it was not available in the material used to prepare this report.

5. Environment Variables and Configuration

Environment variables are configuration values provided to an application at runtime rather than being written directly into the source code. They are useful for storing deployment-specific values such as API base URLs and other configuration settings.

For a deployed application, environment variables help separate development settings from production settings. They also help prevent sensitive values from being unnecessarily exposed in source code.

For MedAssistAI, the frontend needs to communicate with the backend using the appropriate deployed API address rather than a local address such as localhost. Therefore, the frontend configuration must use the backend service available in the deployed environment.

6. Frontend–Backend Communication

Deployment is not complete simply because the frontend and backend are hosted online. Both services must also be able to communicate correctly. The frontend sends requests to backend API endpoints, and the backend returns responses that are displayed by the frontend.

During deployment testing, the important points to verify include the API address, request methods, request data, response handling, authentication information where applicable, and cross-origin communication between the deployed frontend and backend.

The frontend modules developed in the earlier milestone, including Disease Prediction, Risk Assessment, Reports, and role-based dashboards, were designed with backend integration in mind. The earlier milestone prepared these interfaces for future backend integration. This milestone documents the transition toward the deployed and integrated application.

7. Testing the Deployed Application

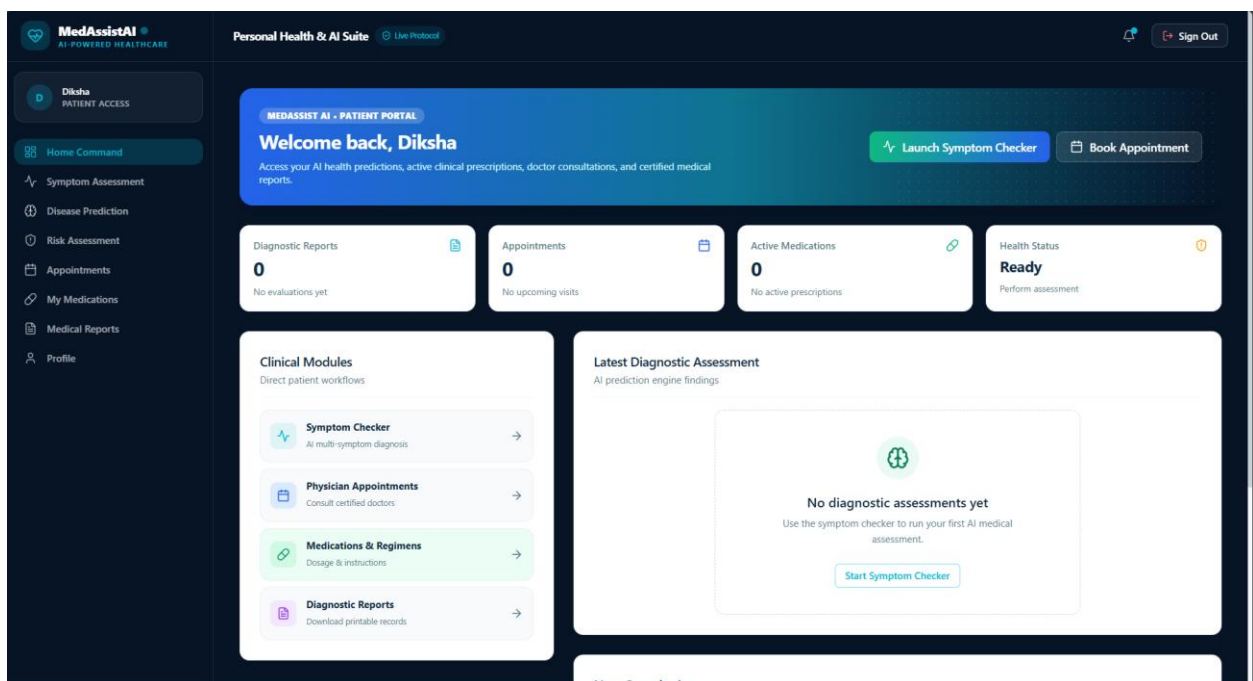
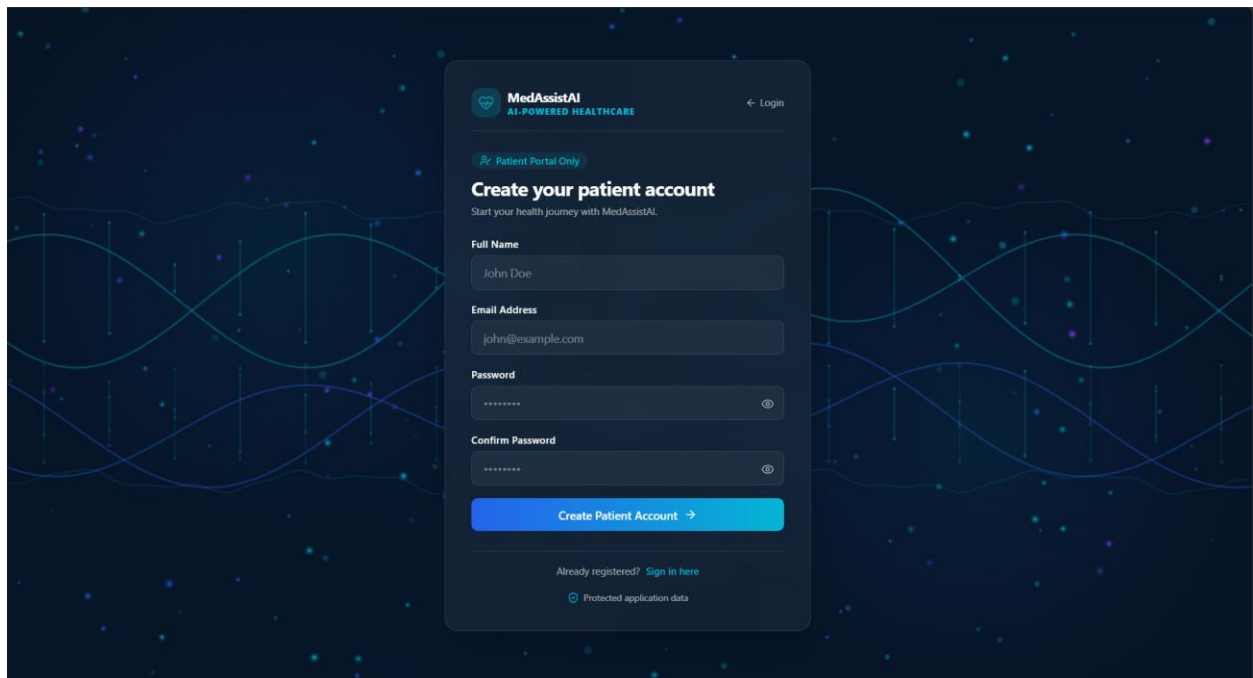
After deployment, the application should be checked from the user's point of view to ensure that the production version is accessible and that the major frontend flows work correctly.

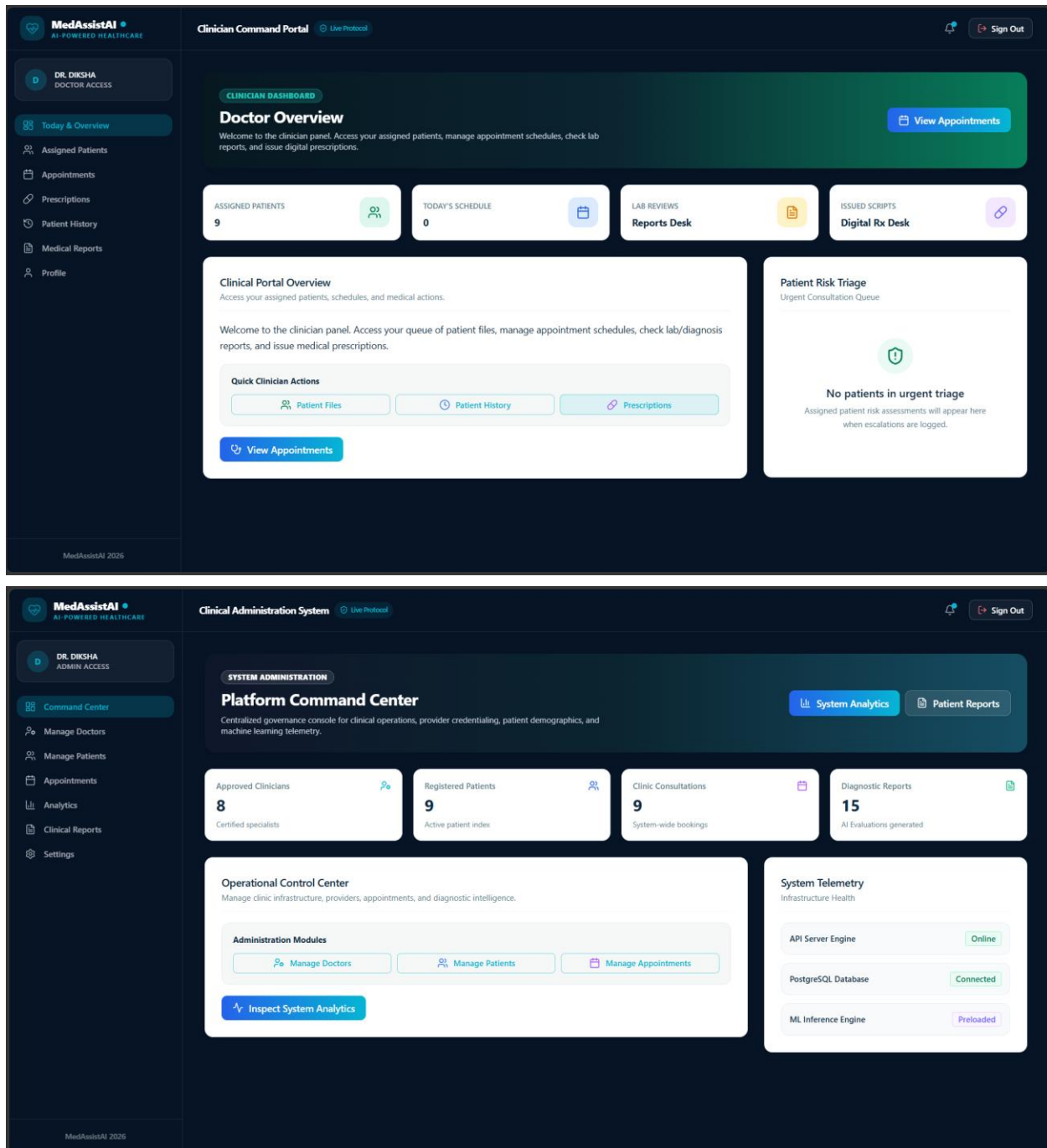
Basic Deployment Test Cases

- Open the deployed frontend URL in a web browser.

- Verify that the application loads without a local development server.
- Check that the main navigation and available pages open correctly.
- Verify the login or registration interface where applicable.
- Check that the frontend can communicate with the deployed backend.
- Verify the major project modules available in the deployed version.
- Check that errors are handled without breaking the user interface.

The following screenshots provide evidence of the deployed application's main user flows and role-based interfaces.





8. Deployment Evidence

The screenshots included in this document show the application interfaces accessed from the deployed project. Together with the public Vercel URL, they provide evidence that the frontend is available in the deployment environment and that the major role-based interfaces can be accessed.

Live Frontend: <https://medassistai-frontend.vercel.app>

9. My Contribution in Milestone 4

My contribution to Milestone 4 was focused on the frontend side of deployment and integration. I studied the deployment workflow and the requirements for moving the frontend from local development to a production environment.

- Reviewed the frontend structure for deployment readiness.
- Understood the purpose of frontend and backend deployment platforms.
- Studied how the frontend communicates with the deployed backend through APIs.
- Verified the accessibility of the team's deployed frontend.
- Reviewed the major frontend modules in the deployed environment.
- Documented the deployment process and the concepts involved in production deployment.

The actual deployment was completed as a team activity. My role in this milestone was primarily focused on understanding, documenting, and verifying the frontend side of the deployment.

10. Learning Outcome

Through this milestone, I learned the basic workflow involved in deploying a web application and understood the difference between a local development environment and a production environment.

I also learned how frontend and backend services can be hosted separately, how APIs connect these services, why environment variables are useful during deployment, and why deployment testing is necessary after moving an application to a production environment.

11. Conclusion

Milestone 4 focused on the deployment and production readiness of the MedAssistAI project. The team deployed the frontend using Vercel and the backend using Render. The deployment provides a production environment in which the application can be accessed and tested outside the local development setup.

From the frontend perspective, this milestone helped me understand how a developed interface is prepared for deployment, how it communicates with a separately deployed backend, and how a live URL can be used to verify that the application is accessible. The milestone also provided practical knowledge of deployment concepts that can be applied to future web projects.